# Side-Channel Attacks on Post-Quantum Cryptography Implementations: A Survey of Attacks, Countermeasures, and Open Challenges

Aditi Bhange, Pranjal Shinde, Saanvi Pathak, Tanishka Kumar
*Department of Computer Engineering and Technology*
*Dr. Vishwanath Karad MIT World Peace University*
Pune, India

***Abstract*—With the standardization of PQC (Post-Quantum Cryptography) standards, namely ML-KEM (FIPS 203), ML-DSA (FIPS 204), SLH-DSA (FIPS 205), and Falcon implemented as FN-DSA (draft FIPS 206), the evaluation of physical security in terms of Side-Channel Analysis (SCA) and Fault Injection Attacks (FIA) is crucial. Even though the mathematical assumptions, such as M-LWE (Module Learning With Errors) and hash-trees, are quantum resistant, their physical implementation can be vulnerable to attacks based on power analysis, dynamic timing leakage, floating point side-channels, and fault-based key recoveries. This paper provides a systematic study of the physical layer security of both standardized and draft PQC algorithms based on the published attacks and countermeasures for ARM Cortex-M4 and FPGA implementations. We define our scope of work in contrast to other surveys, differentiating between instruction-level variable-time execution (for example, KyberSlash) and microarchitectural cache channels. In addition, we provide taxonomy tables for attack techniques, countermeasures, and performance overheads. We also discuss open research challenges in SCA/FIA defense, automation of masking toolchains, and stateless signature schemes.**

***Index Terms*—Post-Quantum Cryptography, Side-Channel Analysis, Fault Injection Attacks, Masking, KyberSlash, ML-KEM, ML-DSA, Falcon, FN-DSA, SLH-DSA.**

## I. Introduction

The move to quantum-resistant cryptography represents a significant change to the public key infrastructure. The standardization of the NIST standards, ML-KEM (FIPS 203), ML-DSA (FIPS 204), and SLH-DSA (FIPS 205), together with Falcon, which is currently being standardized as FN-DSA (draft FIPS 206), relies on non-standard mathematical primitives, including polynomial operations in quotient rings $\mathbb{R}_q = \mathbb{Z}_q[X]/(X^n + 1)$, Fast Fourier Sampling on lattices, and hash tree evaluations. Although all of these protocols have provable security against both classical and quantum algorithmic attacks, the physical implementations on edge microcontrollers, smart cards, and FPGAs leak secret information via power consumption, electromagnetic emissions, time variations, and faulty output.

Security of PQC implementations needs an analysis of the performance of mathematical primitives on physical hardware. Mathematical primitives such as Centered Binomial Distribution (CBD) sampling, Number Theoretic Transforms (NTT), Fujisaki-Okamoto (FO) transform re-encryptions, and floating-point arithmetic provide different physical leakage sources.

While the general surveys on PQC focus on high-level attacks, this research paper provides a review of physical security results from 2018 to 2026, highlights technical differences in side-channel literature, and establishes attack-defense relations.

### A. Contributions

The key contributions of this paper are:

- **Comprehensive Scheme Coverage:** Physical security is assessed for lattice-based key encapsulation schemes (ML-KEM/Kyber, Saber), lattice-based digital signatures (ML-DSA/Dilithium, Falcon/FN-DSA), and hash-based digital signatures (SLH-DSA/SPHINCS+).
- **Distinction from Prior Work:** We compare our scope against existing surveys (e.g., Ravi and Chattopadhyay [2]; Khalid et al. [3]) and highlight our focus on multi-scheme mapping and hardware-software trade-offs.
- **Technical Clarifications:** We define timing vulnerabilities resulting from variable-time instructions (such as integer division in KyberSlash) in comparison to cache probing attacks.
- **Systematic Attack-to-Defense Mapping:** We build comparative matrices linking attack types to their respective defenses, platform cost, and performance overhead.

## II. Background and Mathematical Preliminaries

### A. Lattice-Based Primitives (ML-KEM, ML-DSA, Saber)

Lattice-based standards operate over polynomial rings $\mathbb{R}_q = \mathbb{Z}_q[X]/(X^n + 1)$, where $n = 256$ and $q$ is a prime ($q = 3329$ for ML-KEM; $q = 8380417$ for ML-DSA). Security relies on the hardness of Module Learning With Errors (M-LWE) and Module Short Integer Solution (M-SIS) problems. Polynomial multiplication is accelerated via the Number Theoretic Transform (NTT), reducing $\mathcal{O}(n^2)$ coefficient multiplications to $\mathcal{O}(n \log n)$ point-wise multiplications using the Cooley-Tukey or Gentleman-Sande butterfly operation:

$$y_i = x_i + \zeta \cdot x_j \pmod q, \quad y_j = x_i - \zeta \cdot x_j \pmod q \quad (1)$$

where $\zeta = \omega^\gamma \pmod q$ denotes the primitive root of unity twiddle factor. In unprotected implementations, high-performance modular reductions (such as Montgomery or

Barrett reduction) can introduce secret-dependent Hamming-weight power leakage during intermediate coefficient reduction.

Secret keys are sampled from short-bounded distributions using Centered Binomial Sampling ($\beta_\eta$). Decapsulation in ML-KEM utilizes the Fujisaki-Okamoto (FO) transform to achieve IND-CCA2 security by re-encrypting the decrypted message and validating it against the received ciphertext.

### B. Floating-Point Lattice Signatures (Falcon / FN-DSA)

Falcon (FN-DSA) relies on the NTRU lattice problem over $\mathbb{Z}[X]/(X^n+1)$ and uses Fast Fourier Sampling over lattice trees to generate signatures. Unlike ML-DSA, Falcon signature generation uses floating-point operations to sample from a discrete Gaussian distribution over lattices:

$$D_{\Lambda,\sigma,\mathbf{c}}(\mathbf{x}) \propto \exp\left(-\frac{\|\mathbf{x}-\mathbf{c}\|^2}{2\sigma^2}\right) \tag{2}$$

The computational methods used in this particular case use FFT trees and Gram-Schmidt orthogonalization. Floating point is more difficult to work with than integer values for both constant-time evaluation and blinding; floating point has already been attacked in publications through side channels. (Section V-C).

### C. Stateless Hash-Based Signatures (SLH-DSA / SPHINCS+)

SLH-DSA (SPHINCS+) relies solely on the collision resistant and second-preimage resistant attributes of cryptographic hash functions (like SHA-2 and SHAKE). Key generation is done in a multi-layered hypertree format that combines Winternitz One-Time Signatures (WOTS+) with Forest-of-Random-Subtrees (FORS). The security of the algorithm is based on state non-reuse while generating signatures.

## III. Literature Survey and Related Work

Post-quantum algorithms with physical layer security have gone from theoretical adaptation of classic techniques to specialized research. Table I categorizes representative works according to their platform, target primitive, threat model, security assumptions, and practical severity.

### A. Positioning Relative to Existing Surveys

Previous surveys, such as Khalid et al. [3], established foundational threat models for passive and invasive attacks on early lattice constructions. Hermelink [13] later consolidated side-channel and fault attacks on modern lattice-based schemes. Ravi and Chattopadhyay [2] presented a survey and experimental evaluation focusing on power analysis and fault attacks for Kyber and Dilithium on microcontrollers.

This work complements and extends these contributions in three ways:

1) **Broader Scope:** In this work, we consider the completed standards FIPS 203, FIPS 204, and FIPS 205, in addition to Falcon (FN-DSA, draft FIPS 206), and we also consider the non-lattice construction SLH-DSA as well as a floating point lattice construction.
2) **Technical Clarifications:** We clarify the distinction between variable-time instruction execution and microarchitectural cache attacks.
3) **Defense Synthesis:** We map attack vectors to defense mechanisms, formal verification tools, and reported performance overheads.

## IV. Research Methodology

This survey follows a structured literature review methodology.

### A. Database Search and Query Strings

Literature searches were conducted through IEEE Xplore, ACM Digital Library, SpringerLink, ScienceDirect, the IACR Cryptology ePrint Archive, and arXiv. The Boolean search string combined post-quantum algorithms with physical attack models:

$$\begin{aligned}\text{Query} = (&\text{“ML-KEM”} \vee \text{“ML-DSA”} \vee \text{“Kyber”}\\ &\vee \text{“Dilithium”} \vee \text{“Falcon”} \vee \text{“FN-DSA”}\\ &\vee \text{“SLH-DSA”} \vee \text{“SPHINCS+”} \vee \text{“Saber”})\\ &\wedge (\text{“side-channel”} \vee \text{“fault injection”}\\ &\vee \text{“differential power analysis”} \vee \text{“masking”})\end{aligned}$$

### B. Inclusion, Exclusion Criteria, and Literature Screening

- **Publication Window:** Primary focus on papers published between 2018 and 2026, with foundational exceptions (e.g., Bayrak et al. [14]) included for historical lineage.
- **Inclusion Criteria:** Peer-reviewed journal and conference papers (e.g., IACR TCHES, ACM/IEEE DAC, SCN, PQCrypto), together with Cryptology ePrint/arXiv preprints (e.g., KyberSlash [4]) that present concrete physical attacks, countermeasures, or implementation benchmarks.
- **Exclusion Criteria:** Purely theoretical cryptanalysis papers (e.g., BKZ lattice reduction) lacking a physical implementation or leakage model.
- **Non-Paper Sources:** For context we also cite the NIST standard [1], one doctoral thesis [13], and one industry technical blog [18].
- **Literature Screening Results:** A preliminary search in the database yielded 342 potential records. After deduplication, a total of 184 unique records were screened through titles and abstracts, and further 68 papers were analyzed using the inclusion criteria. Among them, 22 research papers together with 3 non-paper references mentioned above make up a total of 25 references listed in this review.

## V. Technical Analysis of Attack Vectors

### A. ML-KEM and ML-DSA Vectors

Lattice schemes can suffer from distinct physical leakage mechanisms during key generation, encapsulation/signing, and decapsulation:

- **CBD Sampling Leakage:** Centered Binomial Sampling derives secret coefficients $s_i = \sum_{j=0}^{\eta-1} a_j - \sum_{j=0}^{\eta-1} b_j$ from pseudo-random byte streams. In unprotected implementations, power consumption during byte unpacking and

TABLE I
TAXONOMY OF REPRESENTATIVE PHYSICAL SECURITY LITERATURE IN PQC

| Reference | Target Primitive | Platform | Threat Model | Security Assumption / Vector | Practical Impact & Severity[a] |
|---|---|---|---|---|---|
| Ravi & Chattopadhyay [2] | Kyber, Dilithium | ARM Cortex-M4 | Passive Power / Active Glitch | Single/Multi-trace SCA + Fault Injection | High; Survey and new attacks combining SCA and FIA |
| Khalid et al. [3] | General Lattice Cryptography | FPGA / ASIC | Passive Power / EM | Physical access; unmasked hardware primitives | Medium-High; Broad analysis of passive/active hardware threats |
| Bernstein et al. [4] | Kyber / ML-KEM | ARM (Raspberry Pi 2, Cortex-M4) / x86 | Variable Execution Time | Secret-dependent integer division timings | High; Timing-based secret recovery in Kyber implementations |
| Hermelink et al. [5] | Kyber (NTT) | Embedded Software | Passive Power (SASCA) | Belief propagation against shuffled NTT | High; Shows some shuffling countermeasures give a false sense of security |
| Almeida et al. [6] | ML-KEM | Formal Verification Toolchain | Static Code Analysis | Formally verified implementation (Jasmin) with constant-time guarantees | Medium; Formal guarantees of correctness and absence of timing leaks |
| Wang et al. [7] | Dilithium-2 | ARM Cortex-M4 | Single-Trace Power | Secret key unpacking during signature generation | High; 9% single-trace success (when $t_0$ known), high probability with multiple traces |
| Karabulut & Aysu [8] | Falcon / FN-DSA | ARM Cortex-M4 | Passive EM | Floating-point multiplication in pre-image computation | High; First side-channel attack on Falcon |
| Guerreau et al. [9] | Falcon / FN-DSA | ELMO simulator / ARM Cortex-M4 | Power Analysis (SPA) | Gaussian base sampler output | High; Key recovery via hidden parallelepiped attack |
| Amiet et al. [10] | SPHINCS+ | FPGA | Clock/Voltage Glitch FIA | Fault injection on an FPGA SPHINCS+ implementation | High; Authors report the attack breaks the scheme |
| Genêt [11] | SPHINCS+ / SLH-DSA | Analysis / Simulation | Active FIA Analysis | Faults in XMSS tree computation (grafting-tree attack) | High; Evaluates caching defenses against the attack |
| Xu et al. [12] | Kyber | FPGA | Defense (against CPA/TVLA) | Shuffling of NTT-related leakage points | Medium; Hardware shuffling defense with 8.7% efficiency degradation |

[a] *Note: Practical severity levels represent qualitative evaluations synthesized by the authors based on published trace counts and success rates.*

bit Hamming-weight operations can allow adversaries to recover the secret vector using Differential Power Analysis (DPA).

- **Secret Key Unpacking:** Wang et al. [7] show that leakage while unpacking the secret key during signing in an unprotected Cortex-M4 implementation of Dilithium-2 allows secret key recovery from a single trace with about 9% probability (when the low-order bits $t_0$ of the public key are known), and with high probability when multiple traces are available.
- **FO-Transform Re-encryption Attack:** Decapsulation in ML-KEM utilizes the FO transform for IND-CCA2 security:
  1) The receiver decrypts ciphertext $c$ to recover message $m' = \text{Dec}(sk, c)$.
  2) The receiver re-encrypts $m'$ as $c' = \text{Enc}(pk, m', r')$, where $r' = \mathcal{H}(m' \parallel pk)$.
  3) The receiver verifies $c \stackrel{?}{=} c'$. If true, it returns the shared key $K = \mathcal{K}(m' \parallel c)$; otherwise, a pseudo-random value.

Prior work [2] shows that, with chosen ciphertexts, an adversary can exploit leakage during the re-encryption

TABLE II
CROSS-SCHEME PHYSICAL VULNERABILITY AND CHARACTERISTIC COMPARISON

| Algorithm | Mathematical Family | Dominant Leakage Point | Primary Attack Modality | Defense Complexity[b] |
|---|---|---|---|---|
| **ML-KEM** | Module-LWE (Lattice) | FO-Transform Re-encryption, CBD Sampling | Differential Power Analysis (DPA), Glitch FIA [2] | High (higher-order masking + B2A gadgets) |
| **ML-DSA** | Module-LWE / M-SIS | Secret Key Unpacking, Rejection Sampling | Single-Trace Power [7], Fault Attacks [23], [24] | High (masked unpacking and sampling, fault detection) |
| **Falcon / FN-DSA** | NTRU Lattices | Fast Fourier Sampling (FFT), Floating-Point | Floating-Point EM/Power Leakage [8], [9], [16] | Very High (floating-point arithmetic is hard to mask) |
| **SLH-DSA** | Hash-based Hypertree | WOTS+ Chain Evaluation, Tree Hash Loops | Clock/Voltage Glitch FIA [10], [11], Rowhammer [17] | Moderate (redundant hashing, fault detection) |

[b] *Note: Defense-complexity levels are qualitative estimates by the authors.*

Listing 1. Variable-time vs. constant-time division pattern.

```
// Variable-time division
// (vulnerable to KyberSlash)
uint16_t div_vulnerable(uint32_t x,
                        uint16_t q) {
  // Latency depends on dividend x
  return (uint16_t)(x / q);
}

// Constant-time Barrett reduction
int16_t barrett_constant(int32_t a) {
  // Multiply by fixed scale factor
  int32_t t = (a * 20159) >> 26;
  t = a - t * 3329;
  return (int16_t)t;
}
```

or the ciphertext comparison, or fault the comparison, to learn information about intermediate values without solving the underlying M-LWE problem.

- **Rejection Sampling (potential leak):** ML-DSA signatures accept candidates $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$ only if $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$. In principle, if the validation time depends on secret-correlated values, the number of rejection-loop iterations could reveal properties of the secret; this is a conceptual risk rather than a demonstrated attack in the works we survey.

### B. Timing Leaks: Clarifying KyberSlash vs. Cache Attacks

KyberSlash is sometimes confused with microarchitectural cache probing attacks (such as Prime+Probe and Flush+Reload); yet, KyberSlash works through an entirely different instruction-level timing technique.

As documented by Bernstein et al. [4], KyberSlash exploits secret-dependent division timings in Kyber implementations, which the authors evaluate on ARM platforms including a Raspberry Pi 2 and an Arm Cortex-M4. On Cortex-M cores such as the M3 and M4, the hardware division instructions `SDIV`/`UDIV` take a variable number of cycles (roughly 2 to 12) depending on the operands. Consider the division and reduction patterns in Listing 1. When division operates on secret-dependent intermediate values, the latency leaks information about the dividend without accessing the cache hierarchy.

Compilers can also introduce timing vulnerabilities into code that is constant-time at the source level [4], [18].

### C. Falcon / FN-DSA: Floating-Point Side Channels

Falcon's signature generation uses floating-point arithmetic during Fast Fourier Sampling over Gaussian distributions. Experimental analyses by Karabulut & Aysu [8] and Guerreau et al. [9] show that this introduces significant side-channel risks:

- **Floating-Point Multiplication Leakage:**In an electromagnetic attack, the initial side-channel attacks on the Falcon were based on floating-point multiplication used to compute the pre-image. [8].
- **Gaussian Sampler Leakage:** Guerreau et al. [9] show that simple power analysis can reveal the output of Falcon's base sampler, which they turn into key recovery with a hidden parallelepiped attack; they also improve on [8]. Zhou and Wang [16] report leakage when floating-point values are converted from integers in the sampler, requiring over two orders of magnitude fewer traces than earlier sampler attacks under realistic noise.

### D. SLH-DSA: Fault Injection on Hash Structures

SLH-DSA requires the usage of symmetric hashing, and hence does not have any algebraic structure to leak any information; nevertheless, its hash functions can still get leaked through power analysis attacks. Apart from this vulnerability, it is vulnerable to Fault Injection Attack (FIA):

- **Glitching an FPGA Implementation:** Amiet et al. [10] demonstrate clock and voltage glitching against an FPGA SPHINCS+ implementation and report that the attack breaks the scheme.
- **Grafting-Tree Faults:** Castelnovi et al. [15] showed that faults during the computation of an XMSS tree corrupt its root; the next hypertree layer then signs this faulty root with a reused WOTS+ key, which exposes WOTS+ secret values. With enough such signatures, the attacker can graft a tree of their own choosing and forge signatures. Genêt [11] analyzed this attack with layer-caching and branch-caching countermeasures: branch

caching is ineffective with reasonably sized caches, while layer caching has varying effectiveness.
- **Rowhammer:** Software-only Rowhammer attacks can corrupt the internal state during signing and enable forgery without physical access [17].

## VI. Taxonomy of Countermeasures

Defending PQC implementations against physical-layer attacks requires a multi-layered approach across software, compiler, and hardware levels.

### A. Algorithmic Masking and Conversion Gadgets

Masking splits a sensitive intermediate variable $x$ into $d+1$ random shares $x_0, x_1, \ldots, x_d$ to secure implementations against $d$-th order power analysis:

$$x = x_0 \oplus x_1 \oplus \cdots \oplus x_d \qquad \text{(Boolean Masking)} \tag{3}$$

$$x = (x_0 + x_1 + \cdots + x_d) \bmod q \qquad \text{(Arithmetic Masking)} \tag{4}$$

First-order masked Kyber implementations on Cortex-M4 have been demonstrated in [19], [20]. Converting between Boolean and Arithmetic shares (B2A/A2B) introduces performance overheads in masked PQC implementations, and high-order conversion algorithms have been proposed to reduce this cost [21]. Masking schemes must satisfy formal composability conditions such as $d$-probing security and Probe-Isolating Non-Interference (PINI).

### B. Hiding and Random Shuffling

Hiding breaks the temporal correlation between physical leakage and execution state:

- **NTT Operation Shuffling:** The randomization of the order of butterfly operations in NTT loops decreases the signal-to-noise ratio of the side-channel attack. Efficient hardware designs for fully parallel shuffling structures have been suggested as a solution. [12], [22].
- **Attacking Shuffling with Belief Propagation:** Hermelink et al. [5] showed that belief propagation with a "shuffle node" can learn the shuffling permutation of fine shuffling, so that some earlier shuffling countermeasures may give a false sense of security. Countermeasures that add entropy to the representation of ring elements, such as Redundant Number Representation (RNR), can help mitigate such attacks.

### C. Hardware Redundancy and Fault Detection

Defending against FIA requires detecting active fault attempts before faulty outputs are produced:

- **Spatial and Temporal Redundancy:** Re-executing critical hash iterations or parallelizing hash chains with parity verification can detect transient glitches.
- **Hedging Limits:** Hedged signing mixes the secret key with fresh randomness to harden signatures against fault attacks on deterministic signing, but fault attacks on hedged Dilithium have since been demonstrated [23], [24].

### D. Formal Verification and Toolchain Countermeasures

Jasmin is a language and verified compiler framework for high-assurance cryptographic assembly, and has been used to build a formally verified, constant-time ML-KEM implementation [6]. Analysis tools such as Sleuth [14] verify software power-analysis countermeasures, and tools such as TightProver and MaskComp verify masked implementations under specified leakage models.

### E. Quantitative Performance Overhead Analysis

Implementing side-channel countermeasures increases computational overhead. Table IV gives representative reported impacts of masking on embedded ARM hardware. Note that $d$ denotes the masking order, corresponding to $d+1$ shares, and that the Kyber figure is given in absolute cycles whereas the Saber figures are slowdown factors relative to the unmasked implementation, so the rows are not directly comparable.

## VII. Open Challenges and Future Directions

Despite recent progress, several physical security challenges in PQC remain unresolved:

1) **Low-Cost Combined SCA and FIA Defenses:** Power analysis and fault attacks on lattice schemes are surveyed together in [2], but unified countermeasures that protect against both without multiplicative performance penalties remain an active research area.
2) **Automated PQC Masking Compilers:** While verification tools like Sleuth [14] address software power-analysis countermeasures in general, fully automated, provably secure masking compilers tailored to PQC polynomial rings remain an open challenge.
3) **Fault-Resilient Stateless Signature Schemes:** Protecting SLH-DSA against fault injection without heavy redundant hashing requires new countermeasure paradigms [11].
4) **AI-Driven Side-Channel Attacks:** Deep-learning-assisted power analysis has already been used against PQC, for example in profiled attacks on Dilithium [7]. How far such attacks defeat masked and shuffled implementations remains an open research question.
5) **Transient Execution in PQC Implementations:** Speculative-execution attacks such as Spectre may also affect PQC implementations on processors that speculate; their impact on PQC code is less studied than power and fault attacks.
6) **Hardware-Software Co-Design Extensions:** Custom RISC-V ISA extensions for constant-time modular reduction, polynomial arithmetic, and masked NTT butterflies are a promising direction that could provide hardware-level protection with limited software overhead; their benefit for PQC remains to be evaluated.

## VIII. Conclusion

Protecting post-quantum cryptography requires consideration of physical implementation security in addition to algorithmic hardness. In this review, we focus on side channel and fault

TABLE III
SYSTEMATIC ATTACK-TO-DEFENSE MAPPING MATRIX ACROSS PQC IMPLEMENTATIONS

| **Target Attack Vector** | **Primary Defense Mechanism** | **Implementation Layer** | **Associated Cost[c] & Key References** |
|---|---|---|---|
| DPA/CPA on CBD Sampling | Higher-Order Algorithmic Masking | Software / Assembly | High [19], [20] |
| Variable-Time Division (KyberSlash) | Constant-Time Coding / Dynamic Constant-Time Checks (e.g., Valgrind-based) | Compiler / Software | Low [4], [18] |
| Single-Trace Power on Unpacking | Masked or Hardened Unpacking (suggested) | Software | Moderate [7] |
| Floating-Point Gaussian SCA | Constant-time / masked floating-point or integer-based sampling (open problem; no standard solution yet) | Software | High (estimated) [8], [9], [16] |
| FIA on SLH-DSA (e.g., WOTS+ glitching, grafting trees) | Spatial Redundancy / Double Execution, Caching | Software / Hardware | High [10], [11] |
| SASCA on Shuffled NTT | Redundant Number Representation (RNR), Hardware Shuffling | Algorithmic / Hardware | Low–Moderate [5], [12] |
| Compiler Optimization Leaks | Formal Leakage Verification (e.g., Jasmin) | Toolchain | No runtime cost; increased build/verification time [6], [18] |

[c] *Note: Resource cost levels (Low, Moderate, High) are qualitative estimates by the authors based on computational slowdown and memory footprint; measured overheads reported in empirical studies are given in Table IV.*

TABLE IV
MASKING PERFORMANCE OVERHEAD ON EMBEDDED MICROCONTROLLERS (AS REPORTED)

| **Scheme** | **Order ($d$)** | **Overhead** | **Source** |
|---|---|---|---|
| **Kyber-768 Decap. (cycles)** | $d = 1$ | 3.1M cycles | Bos et al. [19] |
| **Saber (slowdown)** | $d = 1$ | 2.7× | Kundu et al. [25] |
| **Saber (slowdown)** | $d = 2$ | 4.9× | Kundu et al. [25] |
| **Saber (slowdown)** | $d = 3$ | 7.7× | Kundu et al. [25] |

injection threats in NIST PQC standardization ML-KEM, ML-DSA, and SLH-DSA, as well as the draft FN-DSA (Falcon). By specifying the difference between variable-time operations and cache channels, by explaining floating point representation and hash structures, and by connecting attack vectors with their protection schemes, this work provides a roadmap for implementing quantum-safe cryptography physically. Since every protection comes at a price, in practice one should prioritize constant time programming, masking/hiding when there is a threat of power analysis, and fault detection when there is physical access.

## REFERENCES


[1] National Institute of Standards and Technology (NIST), "Module-Lattice-Based Key-Encapsulation Mechanism Standard," *FIPS Publication 203*, Aug. 2024.

[2] P. Ravi and A. Chattopadhyay, "Side-channel and fault-injection attacks over lattice-based post-quantum schemes (Kyber, Dilithium): Survey and new results," *IACR Cryptol. ePrint Arch.*, vol. 2022, p. 737, 2022.

[3] A. Khalid et al., "Physical protection of lattice-based cryptography: Challenges and solutions," in *Proc. ACM Great Lakes Symp. VLSI (GLSVLSI)*, 2018, pp. 365–370.

[4] D. J. Bernstein et al., "KyberSlash: Exploiting secret-dependent division timings in Kyber implementations," *IACR Trans. Cryptogr. Hardw. Embed. Syst. (TCHES)*, 2025 (ePrint 2024/1049). [Online]. Available: https://kyberslash.cr.yp.to/

[5] J. Hermelink, S. Streit, E. Strieder, and K. Thieme, "Adapting belief propagation to counter shuffling of NTTs," *IACR Trans. Cryptogr. Hardw. Embed. Syst. (TCHES)*, vol. 2023, no. 1, pp. 60–88, 2023.

[6] J. B. Almeida et al., "Formally verifying Kyber Episode IV: Implementation correctness," *IACR Cryptol. ePrint Arch.*, Paper 2023/215, 2023.

[7] R. Wang, K. Ngo, J. Gärtner, and E. Dubrova, "Unpacking needs protection: A single-trace secret key recovery attack on Dilithium," *IACR Commun. Cryptol. (CiC)*, vol. 1, no. 3, 2024, doi: 10.62056/a0fh89n4e.

[8] U. Karabulut and A. Aysu, "FALCON Down: Breaking FALCON post-quantum signature scheme through side-channel attacks," in *Proc. 58th ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 691–696.

[9] M. Guerreau, A. Martinelli, T. Ricosset, and M. Rossi, "The hidden parallelepiped is back again: Power analysis attacks on Falcon," *IACR Trans. Cryptogr. Hardw. Embed. Syst. (TCHES)*, vol. 2022, no. 3, pp. 141–164, 2022.

[10] D. Amiet, L. Leuenberger, A. Curiger, and P. Zbinden, "FPGA-based SPHINCS+ implementations: Mind the glitch," in *Proc. 23rd Euromicro Conf. Digital System Design (DSD)*, IEEE, 2020, pp. 229–237.

[11] A. Genêt, "On protecting SPHINCS+ against fault attacks," *IACR Trans. Cryptogr. Hardw. Embed. Syst. (TCHES)*, vol. 2023, no. 2, pp. 80–114, 2023.

[12] D. Xu, K. Wang, and J. Tian, "A hardware-friendly shuffling countermeasure against side-channel attacks for Kyber," *arXiv preprint arXiv:2407.02452*, 2024.

[13] J. Hermelink, "Side-channel and fault attacks in modern lattice-based cryptography," Ph.D. dissertation, Dept. Comput. Sci., Bundeswehr Univ. Munich, Neubiberg, Germany, 2024.

[14] A. G. Bayrak, F. Regazzoni, D. Novo, and P. Ienne, "Sleuth: Automated verification of software power analysis countermeasures," in *Cryptographic Hardware and Embedded Systems (CHES)*, Springer, 2013, pp. 293–310.

[15] L. Castelnovi, A. Martinelli, and T. Prest, "Grafting trees: A fault attack against the SPHINCS framework," in *Post-Quantum Cryptography (PQCrypto)*, Springer, 2018, pp. 165–184.

[16] Y. Zhou and W. Wang, "Every signing leaks: Breaking Falcon via floating-point conversion leakage," *IACR Cryptol. ePrint Arch.*, Paper 2026/2124, 2026.

[17] J. Boy, A. Purnal, A. Pätschke, L. Wilke, and T. Eisenbarth, "SLasH-

DSA: Breaking SLH-DSA using an extensible end-to-end Rowhammer framework," *arXiv preprint arXiv:2509.13048*, 2025.

[18] PQShield, "PQShield plugs timing leaks in Kyber / ML-KEM to improve PQC implementation maturity," blog post, 2024. [Online]. Available: https://pqshield.com/?p=3736

[19] J. Bos, M. Gourjon, J. Renes, T. Schneider, and C. van Vredendaal, "Masking Kyber: First- and higher-order implementations," *IACR Trans. Cryptogr. Hardw. Embed. Syst. (TCHES)*, vol. 2021, no. 4, pp. 173–214, 2021.

[20] D. Heinz, M. J. Kannwischer, G. Land, T. Pöppelmann, P. Schwabe, and A. Sprenkels, "First-order masked Kyber on ARM Cortex-M4," *IACR Cryptol. ePrint Arch.*, Paper 2022/058, 2022.

[21] J.-S. Coron, F. Gérard, S. Montoya, and R. Zeitoun, "High-order table-based conversion algorithms and masking lattice-based encryption," *IACR Trans. Cryptogr. Hardw. Embed. Syst. (TCHES)*, vol. 2022, no. 2, pp. 1–40, 2022.

[22] J.-Y. Park, D. Lee, S. Kim, W. Lee, B. G. Kang, and K. Sakurai, "Fully parallel, one-cycle random shuffling for efficient countermeasure against side channel attack and its complexity verification," *IACR Cryptol. ePrint Arch.*, Paper 2023/1889, 2023.

[23] E. Krahmer, P. Pessl, G. Land, and T. Güneysu, "Correction fault attacks on randomized CRYSTALS-Dilithium," *IACR Cryptol. ePrint Arch.*, Paper 2024/138, 2024.

[24] S. Jendral, "A single trace fault injection attack on hedged CRYSTALS-Dilithium," in *Proc. Workshop on Fault Detection and Tolerance in Cryptography (FDTC)*, IEEE, 2024.

[25] S. Kundu, J.-P. D'Anvers, M. Van Beirendonck, A. Karmakar, and I. Verbauwhede, "Higher-order masked Saber," in *Security and Cryptography for Networks (SCN)*, Springer, LNCS 13409, 2022, pp. 93–116.